

Class01C_Introduction_to_Python

<https://spectrum.ieee.org/the-top-programming-languages-2023>

The Top Programming Languages 2023 > Python and SQL are on top, but old languages shouldn't be forgotten

BY STEPHEN CASS | 29 AUG 2023 | 3 MIN READ | 



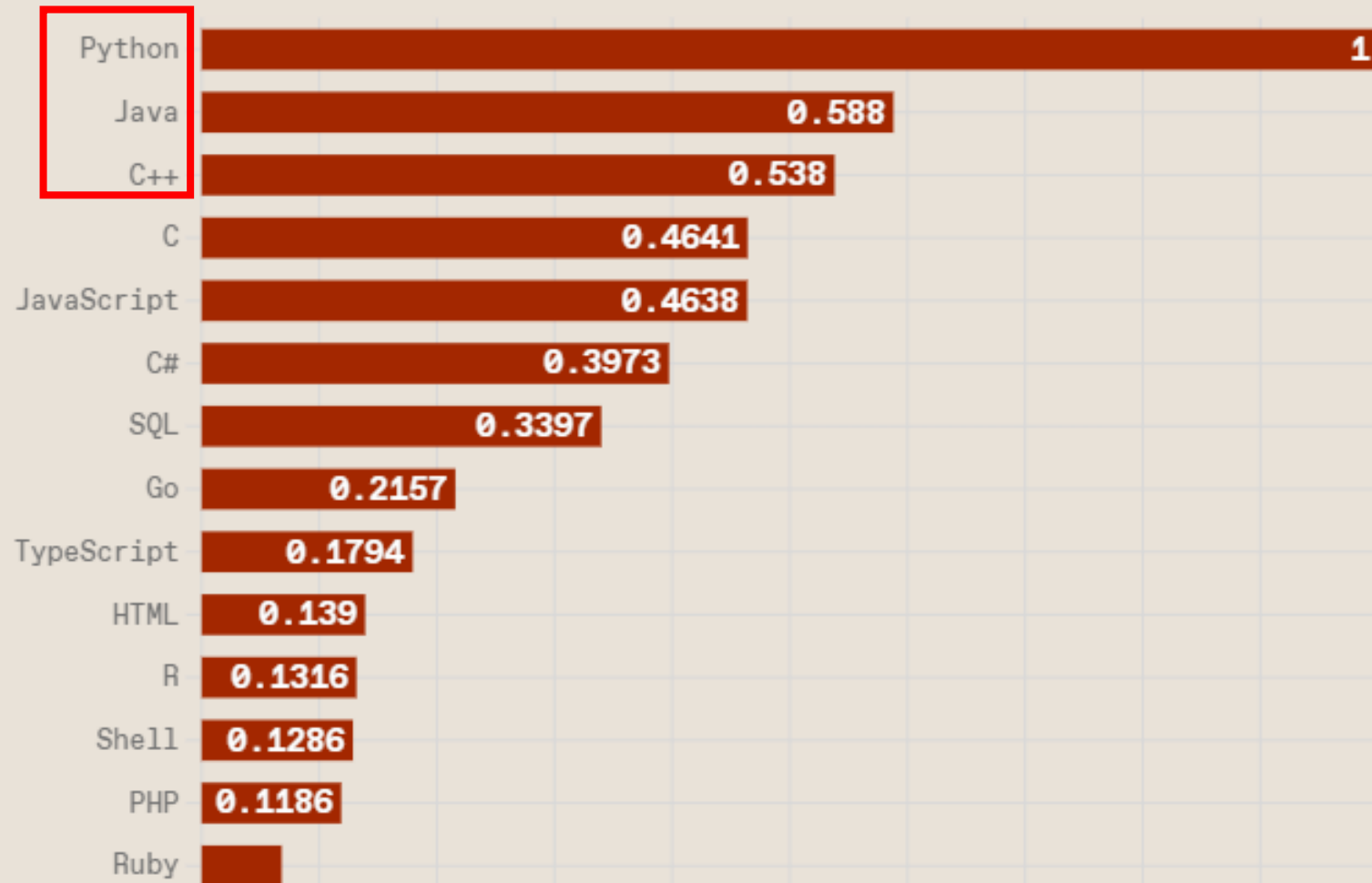
Top Programming Languages 2023

Click a button to see a differently weighted ranking

Spectrum

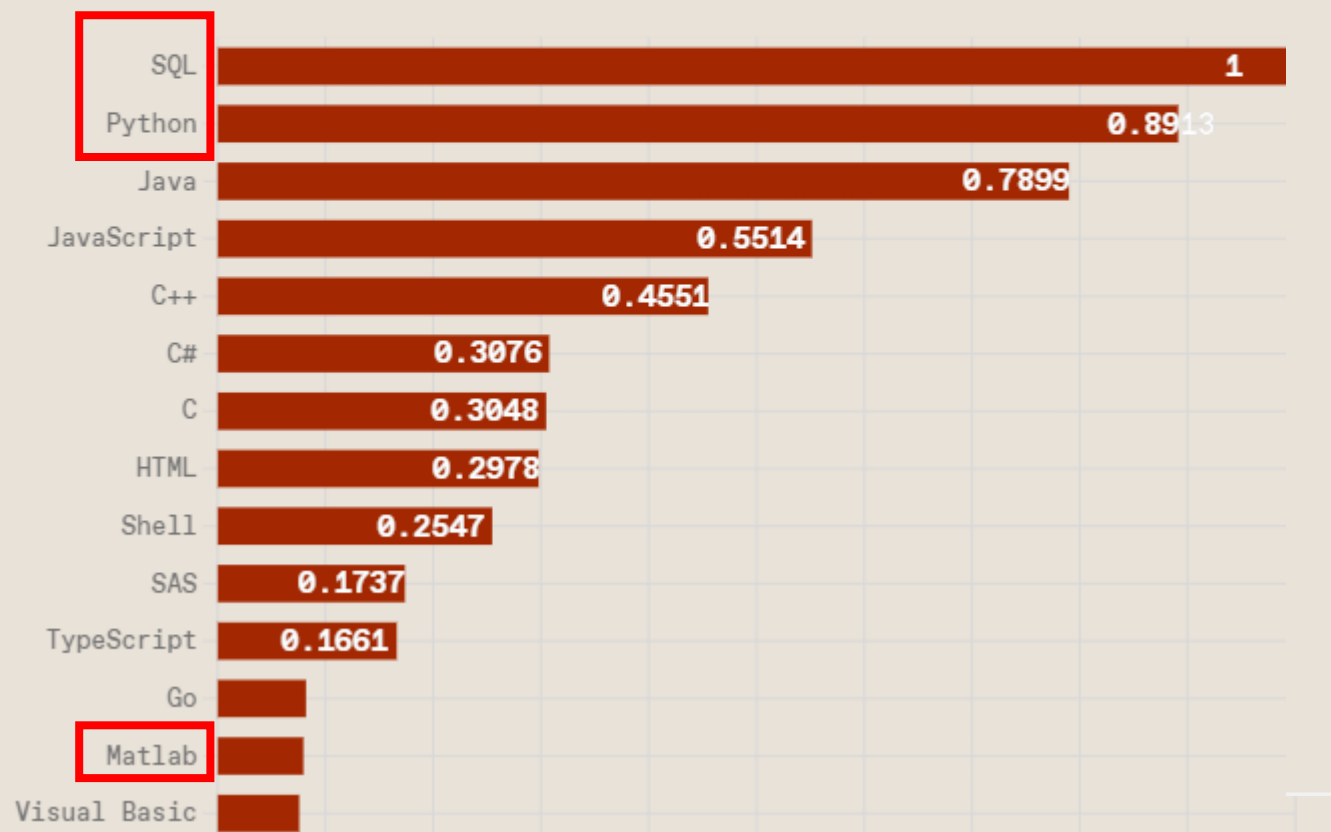
Jobs

Trending



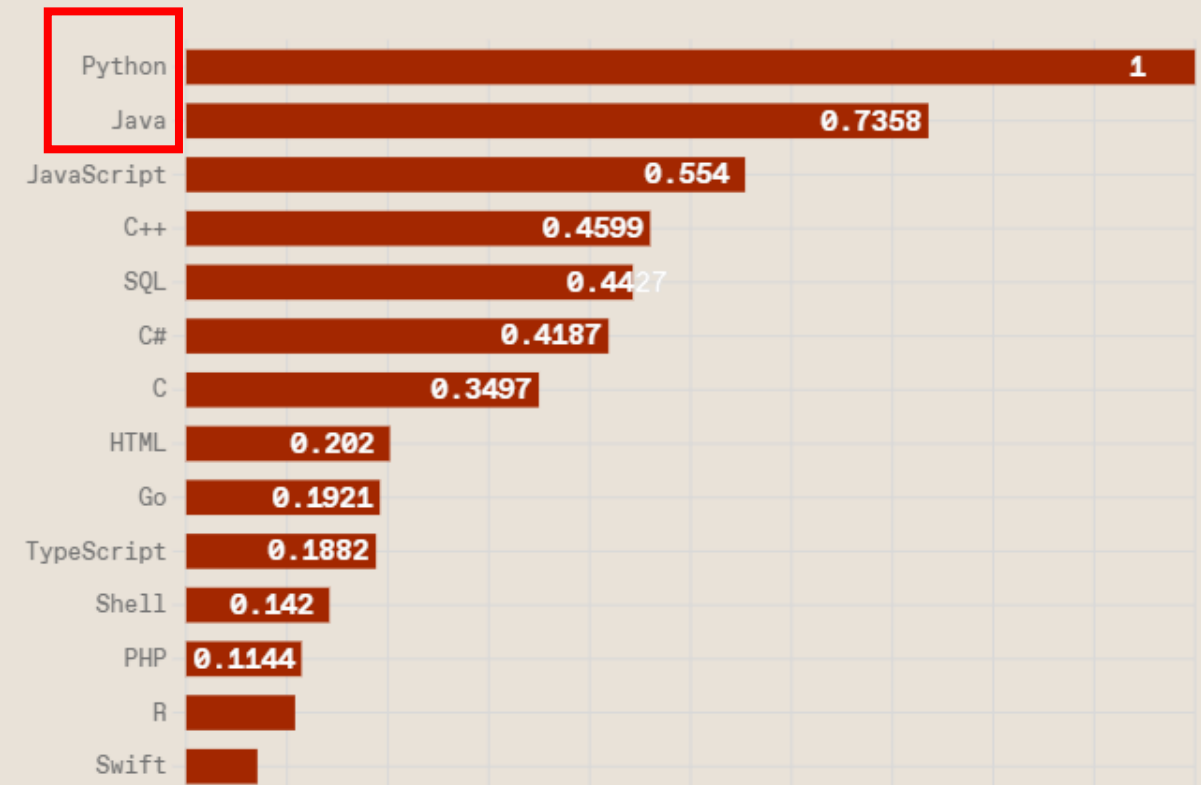
Top Programming Languages 2023

Click a button to see a differently weighted ranking



Top Programming Languages 2023

Click a button to see a differently weighted ranking



Class01C_Introduction_to_Python

How to run: With Colab installed in your Google Drive, move/copy your **.ipynb** files to your Colab folder. **Double click to open !**

The screenshot shows a Google Drive interface with a browser window open to a folder named 'Class4_folder'. The browser's address bar shows the URL 'drive.google.com/drive/folders/1uJ0pLRKguGCK8lugnAreTwTfX9osn_h'. The Google Drive sidebar on the left shows the 'New' button and navigation options like 'My Drive', 'Computers', 'Shared with me', 'Recent', 'Starred', 'Spam', 'Trash', and 'Storage'. The main content area displays a list of files within the 'Class4_folder'. The file 'Class04c_Intro_to_Python.ipynb' is selected and highlighted in blue. Other files in the list include 'Class04e2_Iris_3_keras.ipynb', 'Class04e_Iris_2_sklearn.ipynb', 'Class04g_MNIST.ipynb', 'Class04f_plotdemo_MNIST.ipynb', 'Class04d_Iris_1_PyTorch.ipynb', and 'BCW_dataset.csv'. On the right side, a preview pane shows the selected file 'Class04c_Intro_to_Python.ipynb' with a 'Details' link and a 'Who has access' section indicating it is 'Private to you' with a 'Manage access' button.

Name	Actions
Class04e2_Iris_3_keras.ipynb	⋮
Class04e_Iris_2_sklearn.ipynb	⋮
Class04g_MNIST.ipynb	⋮
Class04f_plotdemo_MNIST.ipynb	⋮
Class04d_Iris_1_PyTorch.ipynb	⋮
Class04c_Intro_to_Python.ipynb	⋮
BCW_dataset.csv	⋮

+ Code + Text

T4 RAM Disk

Double-click (or enter) to edit



Introduction to Python

0s  `if (5>2):`
 `print("Five is greater than two!")`
 Five is greater than two!

Run (this section)

You are beginning to run Colab and to learn python.

Python is a very popular programming language. Python can be used on a server to create web applications, and to connect to database systems. Python can be used to handle big data and perform complex mathematics. **Python** has a large library with many tools. It can be used for rapid prototyping, or for production-ready software development. Libraries such as NumPy, SciPy and Matplotlib allow the effective use of Python in scientific computing. AI libraries are like TensorFlow, Keras, Pytorch and Scikit-learn(sklearn). **Why Python?** Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc). Python has a simple syntax similar to the English language. Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick. Python can be treated in a procedural way, an object-orientated way or a functional way.



+ Code + Text

**Run (this section)**

```
# This is a comment.  
x=5  
y="John"  
# A variable name must start with a letter or the underscore character  
# A variable name cannot start with a number  
# A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _ )  
# Variable names are case-sensitive (age, Age and AGE are different variables)  
x = "awesome"  
print("Python is " + x)  
#Example  
x = 5  
y = 10  
print(x + y)  
print("Sum of x and y =",x+y)
```

```
Python is awesome  
15  
Sum of x and y = 15
```



+ Code + Text

✓ T4 RAM Disk

Python numbers, Strings



Example

`x = 1 # int``y = 2.8 # float``z = 1j # complex``print(type(x)) # To verify the type``y = int(2.8) # y will be 2, python casting``z = str(3.0) # z will be '3.0'``# String literals in python are surrounded by either single quotation marks, or double quotation marks.``# 'hello' is the same as "hello".`

Example

`a = "Hello, World!"``print(a[1]) #get the character at position 1, which is 'e'``print(len(a)) #len() method returns the length of a string``print(a.lower()) #lower() method returns the string in lower case``<class 'int'>``e``13``hello, world!`

Python Operators

▶ # The split() method splits the string into substrings if it finds instances of the separator:

```
b = a.split(",")  
print(b)
```

Command-line String input Example with demo_string_input.py :

```
print("Enter your name:")
```

```
x = input()
```

```
print("Hello, " + x)
```

```
"""Python divides the operators in the following groups:
```

```
Arithmetic operators ; Assignment operators ; Comparison operators
```

```
Logical operators; Identity operators; Membership operators; Bitwise operators"""
```

```
thislist = ["apple", "banana", "cherry"]
```

```
# A python list is created above
```

```
print(thislist)
```

```
print(thislist[1]) #banana will be printed
```

```
thislist[1] = "blackcurrant" #To change the value of a specific item
```

```
print(thislist)
```

```
['Hello', ' World!']
```

```
Enter your name:
```

```
DR. PANG
```

```
Hello, DR. PANG
```

```
['apple', 'banana', 'cherry']
```

```
banana
```

```
['apple', 'blackcurrant', 'cherry']
```



+ Code + Text

✓ T4

Python Loop Through a List

list manipulation

```
[ ] thislist = ['apple', 'blackcurrant', 'cherry']
for x in thislist:
    print(x)                #for loops
if "apple" in thislist:
    print("Yes, 'apple' is in the fruits list") #Check if "apple" is present in the list
print(len(thislist)) #Print the number of items in the list
thislist.append("orange") #Using the append() method to append an item
thislist.insert(1, "peach") #Insert an item as the second position
print(thislist)
thislist.remove("cherry") #removes the specified item
del thislist[0]           #The del keyword removes the specified index
print("To display the list again:", thislist)
thislist.clear()          #The clear() method empties the list to [ ]
thislist = list(("car", "bus", "taxi"))
                        # note the double round-brackets to make a list
print("A new list of items",thislist)
```

```
apple
blackcurrant
cherry
Yes, 'apple' is in the fruits list
3
['apple', 'peach', 'blackcurrant', 'cherry', 'orange']
To display the list again: ['peach', 'blackcurrant', 'orange']
A new list of items ['car', 'bus', 'taxi']
```



+ Code + Text

✓ T4 Runtime Diagnostics

Python Dictionaries



```
# A dictionary is a collection which is unordered, changeable and indexed.  
# In Python dictionaries are written with curly brackets, and they have keys and values.  
# Create and print a dictionary:  
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
print(thisdict)
```

```
# You can access the items of a dictionary by referring to its key name, inside square brackets:  
x = thisdict["model"] #The print out is Mustang  
x = thisdict.get("model") #same result from the get() method  
thisdict["year"] = 2018 #Change the "year" to 2018  
print(thisdict)
```

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
```

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 2018}
```



+ Code + Text

✓ T4

RAM

Disk



{x}



```
for x in thisdict:  
    print('print x:',x)
```

#To print all key names in the dictionary

```
for x in thisdict:  
    print('print thisdict[x]:',thisdict[x])
```

#To print all values in the dictionary

```
for x in thisdict.values():  
    print('print x in thisdict.values():',x)
```

#To use the values() function to return values of a dictionary

```
for x, y in thisdict.items():  
    print(x, y)
```

for

#To loop through both keys and values, by using the items()

```
print(len(thisdict))  
|
```

```
print x: brand  
print x: model  
print x: year  
print thisdict[x]: Ford  
print thisdict[x]: Mustang  
print thisdict[x]: 2018  
print x in thisdict.values(): Ford  
print x in thisdict.values(): Mustang  
print x in thisdict.values(): 2018  
brand Ford  
model Mustang  
year 2018
```



+ Code + Text

✓ T4 Di



```
▶ thisdict["color"] = "red"
print(thisdict)          # To add an item to the dictionary
thisdict.pop("model")    # The pop() method removes the item with the specified key name
print(thisdict)
# del thisdict           #The del keyword can also delete the dictionary completely
thisdict.clear()         #clear() empties the dictionary
print(thisdict)          # The printout would be {}
# The dict() Constructor
thisdict = dict(brand="Ford", model="Mustang", year=1964)
# note that keywords are not string literals
# note the use of equals rather than colon for the assignment
print(thisdict)
```

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 2018, 'color': 'red'}
{'brand': 'Ford', 'year': 2018, 'color': 'red'}
{}
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
```

Python If...then ... else ; Indentation Note that python relies on indentation, using whitespace, to define scope. Other programming languages often use curly-brackets.

```

Python supports the usual logical conditions from mathematics:
Equals: a == b
Not Equals: a != b
Less than: a < b
Less than or equal to: a <= b
Greater than: a > b
Greater than or equal to: a >= b
Example """
a = 33
b = 200
if b > a:
    print("b is greater than a")

# The elif keyword is python's way of saying "if the previous conditions were not true,
# then try this condition". The else keyword catches anything which isn't caught by the preceding conditions.
if b > a:
    print("b is greater than a")
elif a == b:
    print("a and b are equal")
else:
    print("a is greater than b")

# One line if statement:
if a > b: print("a is greater than b")
a = 2
b = 330
print("A") if a > b else print("B")

b is greater than a
b is greater than a
B
```

If-then-else

```
▶ i = 1
while i < 6:
    print(i)
    i += 1          #print out 1,2,3,4,5
print('end of part 1')
i = 1

while i < 6:
    print(i)
    if i == 3:
        break
    i += 1          #print out 1,2,3
print('end of part 2')
i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)        # Note that number 3 is missing in the result
```

while loop

```
1
2
3
4
5
end of part 1
1
2
3
end of part 2
1
2
4
5
6
```

for loop

```
for x in "banana":  
    print(x)                    #b,a,n,a,n,a is printed on separate lines  
print('end of part 1')
```



```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)  
    if x == "banana":  
        break                  #apple, banana printed on two separate lines  
print('end of part 2')
```



```
for x in range(6):  
    print(x)                    #printout of the values 0 to 5  
print('end of part 3')
```



```
for x in range(2, 6):  
    print(x)                    #printout of the values 2 to 5  
print('end of part 4')
```



```
for x in range(2, 22, 3):  
    print(x)                    #printout of the values 2,5,8,11,14,17,20
```

```
b  
a  
n  
a  
n  
a  
end of part 1  
apple  
banana  
end of part 2  
0  
1  
2
```



```

for x in range(6):
    print(x)
else:
    print("Finally finished!")
#Print all numbers from 0 to 5, and print a message when the loop has ended
|
adj = ["red", "big", "tasty"]
fruits = ["apple", "banana", "cherry"]
for x in adj:
    for y in fruits:
        print(x, y)          #Printout 9 lines

```

```

0
1
2
3
4
5
Finally finished!
red apple
red banana
red cherry
big apple
big banana
big cherry
tasty apple
tasty banana
tasty cherry

```

Python Functions



"""A function is a block of code which only runs when it is called.
You can pass data, known as parameters, into a function.
A function can return data as a result."""

```
def my_function():  
    print("Hello from a function")  
my_function()    #calling a function  
def my_function(fname):  
    print(fname + " is coming.")
```

```
|  
my_function("Linus")  
# To use a default parameter value
```

```
def my_function(country = "Norway"):  
    print("I am from " + country)
```

```
my_function("Sweden")  
my_function()
```



```
Hello from a function  
Linus is coming.  
I am from Sweden  
I am from Norway
```

Python Arrays



```
cars = ["Ford", "Volvo", "BMW"]  
# Get the value of the first array item:  
x = cars[0]  
# Modify the value of the first array item:  
cars[0] = "Toyota"  
# Return the number of elements in the array:  
print('length of cars:',len(cars))  
# Print each item in the array  
for x in cars:  
    print(x)  
#Add one more element to the array:  
cars.append("Honda")  
#Delete the element that has the value "Volvo":  
cars.remove("Volvo")  
print('The car array is ',cars)
```

```
length of cars: 3  
Toyota  
Volvo  
BMW  
The car array is  ['Toyota', 'BMW', 'Honda']
```

Python Classes and Objects



```
# To create a class, use the keyword class:
class MyClass:
    x = 5
# Now we can use the class named MyClass to create objects:
p1 = MyClass()
print(p1.x)
# All classes have a function called __init__(), which is always executed when the class is being initiated.
# Create a class named Person, use the __init__() function to assign values for name and age:
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("John", 36)
print(p1.name)
print(p1.age)
```

```
5
John
36
```

Python Modules



"""A module is a file containing a set of functions you want to include in your application. To create a module just save the code you want in a file with the file extension .py: Save this code in a file named abc.py"""

```
def greeting(name):  
    print("Hello, " + name)  
# Now we can use the module we just created, by using the import.
```

```
# import abc  
# abc.greeting("Jonathan")
```

```
# Renaming a Module:  
# import abc as mx  
# mx.greeting("John")
```

```
greeting('Grantham')
```



Hello, Grantham



```
# Import the datetime module and display the current date:
import datetime
x = datetime.datetime.now()
print(x)
|
try:
    print(x)
except:
    print("An exception occurred")
else:
    print("Nothing went wrong")

import os
# Get the current directory in Colab and print it out
tmp=os.getcwd()
print(tmp)
# list the files in the current directory
os.listdir()
```

2022-09-30 10:17:45.775382

2022-09-30 10:17:45.775382

Nothing went wrong

/content

['.config', 'sample_data']

To check the execution time

```
import datetime
```

```
start_time = datetime.datetime.now()
```

```
.....
```

```
end_time = datetime.datetime.now()
```

```
runtime = end_time - start_time
```

```
print(f"Program runtime:{runtime}")
```

```
print(f"Program runtime in seconds:{runtime.total_seconds()}")
```

Other Facilities and File Handling

```
▶ #Create a file called "myfile.txt" by using "x":  
f = open("myfile.txt", "x")  
f.write("Now the file has a new line line! ")  
f.write("The file has a second line!")  
f.close  
  
tmp=os.listdir()  
print(tmp)  
  
#To open a file, and use the read() method to read its content:  
f = open("myfile.txt", "r")  
print(f.read())  
  
#Remove the file "myfile.txt"  
os.remove("myfile.txt")  
  
tmp2=os.listdir()  
print(tmp2)  
  
['.config', 'myfile.txt', 'sample_data']  
Now the file has a new line line! The file has a second line!  
['.config', 'sample_data']
```

**Learn to
Speak
Python !**

There are many more things to learn. Use the online search and help. Learn to Speak Python !

Python offers a variety of frameworks for machine learning and deep learning, each tailored to specific use cases. TensorFlow is one of the most popular.

TensorFlow refers to an open-source machine learning platform/framework developed by Google. It provides a comprehensive Python API that allows developers to easily define, train, and deploy machine learning models, particularly deep learning models and neural networks.

Tensors:

- The fundamental data structure in TensorFlow is the "tensor," which is a multi-dimensional array. Operations in TensorFlow are performed on these tensors.

PyTorch

PyTorch, created by Facebook, is another leading deep learning framework. Known for its dynamic computational graph, it allows flexibility during runtime, making it ideal for research and experimentation. PyTorch is widely used in computer vision and NLP tasks, with applications ranging from Tesla's self-driving AI to real-time fraud detection.

Keras

Keras is a high-level neural network API that runs on top of TensorFlow. It is designed for fast experimentation and ease of use. Keras is commonly used in industries like healthcare for predictive diagnostics and in mobile applications for machine learning tasks¹.

Scikit-learn

Scikit-learn is a framework for traditional machine learning tasks such as regression, classification, and clustering. It is built on NumPy, SciPy, and Matplotlib, making it highly efficient for small to medium-sized datasets. Companies like Spotify use Scikit-learn for recommendation systems

Other frameworks: Theano, Deeplearning4j (DL4J), Sonnet.